

Future Course Material

Page Content

- [Current Status](#)
- [Ongoing Efforts](#)
- [Course Proposal](#)
 - [Introduction to the seL4 microkernel \(TODO days\)](#)
 - [Data61 seL4 Tutorials](#)
 - [UNSW AOS Course](#)
 - [Introduction to the CAMkES framework \(TODO days\)](#)
 - [TRENTOS basic course \(3 days\)](#)
 - [TRENTOS advanced course \(3 days\)](#)
 - [TRENTOS driver development course \(4 days\)](#)
 - [TRENTOS professional course - Industrial Automation \(5 days\)](#)
 - [TRENTOS professional course - Autonomous Driving \(5 days\)](#)
- [Train the trainer - Course Description](#)
 - [Basic Course \(80 hours\)](#)
 - [Course Timeline](#)
 - [Extended Course \(80 hours\)](#)
 - [Practical Project 1 - Industrial Automation](#)
 - [Practical Project 2 - Autonomous Driving](#)
 - [Course Timeline](#)
- [Tasks](#)

HENSOLDT Cyber wants to offer course material, covering both the seL4 ecosystem (consisting of seL4 and CAMkES) in general and TRENTOS in specific. Currently, three different directions exist:

- university cooperation, e.g. TRENTOS TUM course (1 semester)
 - [SEOS-1681](#) - Getting issue details... [STATUS](#)
 - [DPB-57](#) - Getting issue details... [STATUS](#)
 - [DPB-70](#) - Getting issue details... [STATUS](#)
- seL4 Foundation
 - seL4/CAMkES training (2-3 days)
 - [SEOS-1410](#) - Getting issue details... [STATUS](#)
 - [SEOS-1411](#) - Getting issue details... [STATUS](#)
 - TRENTOS training (2-3 days)
 - [SEOS-1410](#) - Getting issue details... [STATUS](#)
 - [SEOS-1411](#) - Getting issue details... [STATUS](#)
- train the trainer (e.g. Indiahub)
 - basic course (80 hours)
 - extended course (80 hours)

Current Status

Dedicated to university cooperations, a basic TRENTOS training has already been created in form of the [TRENTOS TUM course](#). With respective adaptations, the existing course material can be prepared to also cover

- the TRENTOS training (seL4 Foundation)
- the basic course (train the trainer)

Based on the team project task the students in the TUM course are working on, also material covering an extended course (train the trainer) could be created. Regarding the seL4/CAMkES training (seL4 Foundation) we nevertheless have to create new material, as seL4 and CAMkES specific tutorials have not been part of the current course efforts.

Ongoing Efforts

In a first step, it was planned to adapt the existing TUM course material for providing it to the seL4 Foundation ([SEOS-2175](#) - Getting issue details... [STATUS](#)). In parallel, also the provision of "train the trainer" is planned ([STR-12](#) - Getting issue details... [STATUS](#)).

Course Proposal

As the three directions presented above tend to focus on a rather broad range, which might not suit every potential customer due to differing individual background knowledge, the following approach provides a proposal for a more fine-grained course offering. Each course is thereby planned for a duration of 3-5 days in total. The courses are designed modular, so that people can choose a specific topic based on their previous knowledge or start right from the beginning to improve their knowledge step by step. The following table provides a respective overview about the individual courses, the corresponding course content as well as the potential target audience:

Course	Content	Target Audience
Introduction to the seL4 microkernel (TODO days)	- theoretical foundation of the seL4 microkernel (lecture units) - deep dive into the seL4 kernel API (practical work) - development of basic OS facilities directly on top of seL4 (practical work)	Beginner/Advanced - principal knowledge of computer architecture - principal knowledge of operating systems - basic experience in kernel design - experience in programming with the C programming language
Introduction to the CAMkES framework (TODO days)	- basics of the seL4 microkernel (lecture unit) - theoretical foundation of the CAMkES framework (lecture units) - development of basic OS facilities based on the CAMkES abstraction layer (practical work)	Beginner/Advanced - basic understanding of computer architecture - principal knowledge of operating systems - basic understanding of the seL4 microkernel - experience in programming with the C programming language
TRENTOS basic course (3 days)	- basics of the seL4 microkernel and the CAMkES framework (lecture units) - theoretical foundation of TRENTOS (lecture unit) - Tutorial: How to write a first TRENTOS application (practical work) - Storage - File System - Raspberry Pi 3 B+	Beginner - basic understanding of computer architecture - principal knowledge of operating systems - basic understanding of the seL4 microkernel and the CAMkES framework - experience in programming with the C programming language
TRENTOS advanced course (3 days)	- Advanced practical assignments on the core TRENTOS topics - Storage - Network - Crypto	Advanced requires the TRENTOS basic course level
TRENTOS driver development (4 days)	- Device driver integration in TRENTOS - Practical assignments regarding TRENTOS device driver development, by exemplarily using SPI-based peripherals - SPI-based FRAM adapter - SPI-based Ethernet adapter	Advanced requires the TRENTOS advanced course level

TRENTOS professional course - Industrial Automation (5 days)	• Application of TRENTOS to a real-world scenario (autonomous sorting) from the industrial automation domain, based on the open source robotic simulator CoppeliaSim • A TRENTOS application running on the Raspberry Pi 3 B+ ◦ reads virtual sensor data from the simulator ◦ calculates respective actuator output ◦ sends actuator data to the simulator	Advanced • requires the TRENTOS advanced course level
TRENTOS professional course - Autonomous Driving (5 days)	• Application of TRENTOS to a real-world scenario (autonomous parking) from the automotive domain, based on the open source autonomous driving simulator CARLA • A TRENTOS application running on the Raspberry Pi 3 B+ ◦ reads virtual sensor data from the simulator ◦ calculates respective actuator output ◦ sends actuator data to the simulator	Advanced • requires the TRENTOS advanced course level

Introduction to the seL4 microkernel (TODO days)

Currently available resources are provided via:

- seL4 Tutorials provided by Data61: <https://docs.sel4.systems/Tutorials/>
 - <https://github.com/sel4/sel4-tutorials-manifest>
 - <https://github.com/sel4/sel4-tutorials>
- UNSW Advanced Operating Systems (AOS) course: <https://www.cse.unsw.edu.au/~cs9242/20/lectures.shtml>
- HS-RM Advanced Operating Systems (AOS) course: https://www.cs.hs-rm.de/~kaiser/2020_aos.php

Data61 seL4 Tutorials

The following topics are currently covered by the seL4 tutorials provided by Data61:

Type	Tutorial	Content	Source
Introduction	Hello World	Simple hello world program running as user-level application on seL4	https://docs.sel4.systems/Tutorials/hello-world.html
seL4 mechanisms (small exercises around using the kernel API)	Capabilities	introduction to capabilities in the seL4 kernel API	https://docs.sel4.systems/Tutorials/capabilities.html
	Untyped	user-level memory management	https://docs.sel4.systems/Tutorials/untyped.html
	Mapping	virtual memory in seL4	https://docs.sel4.systems/Tutorials/mapping.html
	Threads	how to start a thread using the seL4 API	https://docs.sel4.systems/Tutorials/threads.html
	IPC	overview of interprocess communication (IPC)	https://docs.sel4.systems/Tutorials/ipc.html
	Notifications	using notification objects and signals	https://docs.sel4.systems/Tutorials/notifications.html
	Interrupts	receiving and handling interrupts	https://docs.sel4.systems/Tutorials/interrupts.html
	Faults	fault (e.g virtual memory fault) handling and fault endpoints	https://docs.sel4.systems/Tutorials/fault-handlers.html
Rapid prototyping (walkthroughs and exercises for using the dynamic libraries provided in seL4_libs)	Dynamic Libraries 1	system initialisation & threading with seL4_libs	https://docs.sel4.systems/Tutorials/dynamic-1.html
	Dynamic Libraries 2	IPC with seL4_libs	https://docs.sel4.systems/Tutorials/dynamic-2.html
	Dynamic Libraries 3	process management with seL4_libs	https://docs.sel4.systems/Tutorials/dynamic-3.html
	Dynamic Libraries 4	timers and timeouts with seL4_libs	https://docs.sel4.systems/Tutorials/dynamic-4.html
MCS extensions	MCS	an introduction to the seL4 MCS extensions	https://docs.sel4.systems/Tutorials/mcs.html

UNSW AOS Course

The UNSW AOS course covers the following topics within the lecture units:

Lecture	Content	Source
Introduction (Microkernels and seL4)		https://www.cse.unsw.edu.au/~cs9242/20/lectures/01a-intro.pdf
seL4 API and usage		https://www.cse.unsw.edu.au/~cs9242/20/lectures/01b-sel4.pdf
Microkernel Design and Implementation (with focus on seL4)		https://www.cse.unsw.edu.au/~cs9242/20/lectures/05b-uk.pdf
Performance Measurement and Analysis		https://www.cse.unsw.edu.au/~cs9242/20/lectures/04b-perf.pdf
Formal Verification and seL4		https://www.cse.unsw.edu.au/~cs9242/20/lectures/08a-sel4.pdf
Local OS Research (Having fun with seL4 and beyond)		https://www.cse.unsw.edu.au/~cs9242/20/lectures/10b-local.pdf

Lecture recordings can be found at <https://www.youtube.com/playlist?list=PLbSaCpDlfd6qLbEsKquVo3--0gwYBmrUV>

The UNSW AOS course covers the following topics within the practical assignments, using the Odroid C2 SBC as hardware platform:

Assignment	Content	Source
M0: Familiarisation	Familiarisation with the provided source code and build system and development of a simple IPC protocol.	https://www.cse.unsw.edu.au/~cs9242/20/project/m0.shtml
M1: A timer driver	Development of a simple device driver for the timers available on the Odroid C2.	https://www.cse.unsw.edu.au/~cs9242/20/project/m1.shtml
M2: System call interface	Development of the system call interface for the operating system.	https://www.cse.unsw.edu.au/~cs9242/20/project/m2.shtml
M3: A virtual memory manager	Development of a virtual memory management system to manage mapping and fault handling of processes within the operating system.	https://www.cse.unsw.edu.au/~cs9242/20/project/m3.shtml
M4: The filesystem	Development of filesystem related system calls based on provided code. Additional usage of a timer driver to benchmark the implementation.	https://www.cse.unsw.edu.au/~cs9242/20/project/m4.shtml
M5: Demand paging	Implementation of demand paging in the operating system.	https://www.cse.unsw.edu.au/~cs9242/20/project/m5.shtml
M6: Process management	Development of process loading and management (implementing the required system calls)	https://www.cse.unsw.edu.au/~cs9242/20/project/m6.shtml
M7: Documentation and final system	Completion of the documentation for the project.	https://www.cse.unsw.edu.au/~cs9242/20/project/m7.shtml

Introduction to the CAMkES framework (TODO days)

Currently available resources are provided via:

- CAMkES Tutorials provided by Data61: <https://docs.sel4.systems/Tutorials/>
 - <https://github.com/sel4/sel4-tutorials-manifest>
 - <https://github.com/sel4/sel4-tutorials>

The following topics are currently covered by the CAMkES tutorials provided by Data61:

Type	Tutorial	Content	Source
Introduc tion	CAMkES	an introduction to CAMkES concepts, introducing the CAMkES syntax, bootstrapping a basic static CAMkES application and describing its components	https://docs.sel4.systems/Tutorials/hello-camkes-0.html
	CAMkES 1	an introduction to CAMkES: bootstrapping a basic static CAMkES application, describing its components, and linking them together	https://docs.sel4.systems/Tutorials/hello-camkes-1.html
	CAMkES 2	an introduction to CAMkES: bootstrapping a basic static CAMkES application, describing its components, and linking them together	https://docs.sel4.systems/Tutorials/hello-camkes-2.html
	CAMkES 3	guides through setting up a sample timer driver component in CAMkES and using it to delay for 2 seconds using the ZYNQ7000 ARM-based platform	https://docs.sel4.systems/Tutorials/hello-camkes-timer.html
Virtual Machin es	CAMkES VM Linux	an introduction to creating VM guests and applications on seL4 using CAMkES	https://docs.sel4.systems/Tutorials/camkes-vm-linux.html
	CAMkES Cross-VM communication	an introduction to using the cross virtual machine (VM) connector mechanisms provided by seL4 and Camkes in order to connect processes in a guest Linux instance to Camkes components	https://docs.sel4.systems/Tutorials/camkes-vm-crossvm.html

TRENTOS basic course (3 days)

This course introduces the fundamental aspects of TRENTOS (<https://www.trentos.de/>), a novel seL4 microkernel based secure embedded operating system developed by HENSOLDT Cyber (<https://hensoldt-cyber.com/>). The course is split in a lecture part, which covers the theoretical background, and a practical part that provides a basic case in order to teach participants how to create their very first TRENTOS application. Finally, the theoretical knowledge is checked in form of a multiple choice test.

For providing a solid theoretical foundation, the first part of the course covers the basic aspects required for embedded computing in general, providing an introduction to fundamental hardware aspects of the ARM architecture, its respective ISA as well as the final integration of a processor within a system on a chip (SoC). Based on the Broadcom BCM2837, the integration and utilization of a SoC is then exemplarily demonstrated by utilizing the widespread Raspberry Pi 3 B+ single-board computer (SBC). Afterwards, the relevant preparation steps required for further OS utilization are addressed. Relevant aspects include the working with technical reference manuals (TRMs) as well as the utilization of relevant firmware and bootloader components (e.g. U-Boot). Finally, TRENTOS, a secure L4 microkernel based operating system for embedded system development, is introduced. Therefore, the design, architecture and the essential building blocks of the seL4 microkernel are explained. Fundamental aspects like inter-process communication (IPC), the core concept of capabilities, system calls or memory and I/O management are presented in more detail. Based on top of seL4, the CAmkES framework provides respective kernel abstractions that are helpful for further OS and application development, allowing the provision of a suitable userland environment. As a final step, the core concepts and basic software modules of TRENTOS, a secure operating system that utilizes both the seL4 microkernel and the CAmkES framework, are presented. Course participants will understand the inner workings of the OS and get in touch with the main OS aspects storage, networking and crypto support. Additional topics covered will target TRENTOS application and driver development.

Based on a small guided practical assignment, course participants will then be taught how to create their first TRENTOS application that can be executed on the Raspberry Pi 3 B+. This will combine a step by step guide (provided and explained by the teacher) and self-contained programming tasks. The basic case will cover the topics

- Basic Case - Part 1: Create a basic TRENTOS application utilizing storage and file system facilities within a virtual execution environment
 - run TRENTOS on QEMU
 - utilize a memory-backed storage backend as a virtual disk
 - provide two partitions
 - understand the TRENTOS storage facilities in combination with the TRENTOS FileSystem API
 - utilize two different file systems (FAT and SPIFFS)
- Basic Case - Part 2: Run TRENTOS on the Raspberry Pi 3 B+
 - understand the TRENTOS build system
 - prepare the SD card resources (e.g. firmware blobs)
 - prepare the wiring/cabling of the SBC
 - receive terminal output from the Raspberry Pi 3 B+ on the PC for debugging purposes

At the end of the course, the knowledge learnt is checked in form of a multiple choice test. The following table provides a detailed overview of the course timeline.

Task	Duration
Lecture	10h
SDK overview	2h
How to write a first TRENTOS application	8h
Multiple Choice Test	2h

TRENTOS advanced course (3 days)

The TRENTOS advanced course offers additional practical training by extending the "How to write your first TRENTOS application" setup provided in the TRENTOS basic course. Three assignments have to be solved, covering the main TRENTOS topics storage, networking and crypto. This will combine a step by step guide (provided and explained by the teacher) and self-contained programming tasks. The assignments will cover the topics

- Assignment 1: Replace the virtual storage backend by a physical storage backend
 - connect a SPI-based NOR flash peripheral device to the Raspberry Pi 3 B+
 - understand the usage of a timer resource
 - understand the concept of simply exchanging underlying storage backends by relying on the abstraction provided by the TRENTOS Storage API
 - reuse the existing TRENTOS SPI NOR flash driver
- Assignment 2: Extend the application via network support utilizing the SBC's internal network interface
 - understand the basic concept of a NIC driver in TRENTOS
 - understand the concept of the TRENTOS network interface
 - understand the utilization of a network stack on top of a NIC driver (both in server and client mode)
 - understand basic network communication on top of the network stack
- Assignment 3: Extend the network part of the application via crypto support
 - understand the basic concept of the TRENTOS Crypto API
 - utilize the TRENTOS TLS API to secure the existing network connection

The following table provides a detailed overview of the course timeline.

Task	Duration
Assignment 1 (Storage)	4h

Assignment 2 (Network)	12h
Assignment 3 (Crypto)	8h

TRENTOS driver development course (4 days)

After having dealt with the TRENTOS application layer, the course participants will get in touch with the device driver level. Within two small projects, the participants will write a device driver for two SPI-based peripherals that shall replace the existing storage and network backends used within the assignments presented in the TRENTOS advanced course. Support shall be provided for

- a SPI-based Ethernet peripheral
- a SPI-based FRAM peripheral

The participants will thereby mainly focus on driver porting, which means that an existing open source driver shall be adapted to the respective TRENTOS interfaces (network and storage) and is finally integrated and tested on TRENTOS. The participants shall reuse the underlying GPIO and SPI basic drivers from the TRENTOS SPI NOR flash component and therefore focus on providing the actual peripheral logic.

The following table provides a detailed overview of the course timeline.

Task	Duration
SPI-based FRAM driver	16h
SPI-based Ethernet driver	16h

TRENTOS professional course - Industrial Automation (5 days)

The TRENTOS professional course with focus on industrial automation builds upon the theoretical knowledge and practical experience acquired within the TRENTOS basic and TRENTOS advanced courses. Therefore, TRENTOS will be applied within the specific real world scenario industrial automation. The test environment is provided with the help of an open source simulator, which provides the required virtual sensors and actuators that have to be read and controlled by a respective TRENTOS application. A TRENTOS instance on the Raspberry Pi 3 B+ shall thereby allow for an autonomous sorting setup. It therefore has to control a robotic arm as well as several conveyor belts provided by a robotic simulator that is running on a Linux PC. The robotic arm receives objects of two different colors via a feeding conveyor belt, reads the color of the objects with the help of a virtual color sensor and finally sorts the objects by color via selection between two continuing conveyor belts. As the arm shall be controlled via TRENTOS, relevant sensor data (e.g. from the color sensor) has to be sent from the simulator to the Raspberry Pi 3 B+. The TRENTOS application then calculates the required actuator output (e.g. for the robotic arm's servo motors) and communicates relevant data back to the PC. In addition, the amount of objects sorted by color shall be kept in a non-volatile storage. The communication between the Raspberry Pi 3 B+ and the PC is based on Ethernet, utilizing the simulator's API on top. Within the practical project setup, the open source robotic simulator [CoppeliaSim](#) will be applied.

The following table provides a detailed overview of the course timeline.

Task	Duration
Practical Project - Industrial Automation	40h

TRENTOS professional course - Autonomous Driving (5 days)

The TRENTOS professional course with focus on autonomous driving builds upon the theoretical knowledge and practical experience acquired within the TRENTOS basic and TRENTOS advanced courses. Therefore, TRENTOS will be applied within the specific real world scenario autonomous driving. The test environment is provided with the help of an open source simulator, which provides the required virtual sensors and actuators that have to be read and controlled by a respective TRENTOS application. A TRENTOS instance on the Raspberry Pi 3 B+ shall allow for an autonomous parking setup. It therefore has to control a vehicle in an autonomous driving simulator that runs on a Linux PC. The car drives on a road passing vehicles of different size and distance to each other and measures the distance between itself and the vehicles parking on the road in order to detect free spaces. If a free space is detected, its size has to be measured and compared to the size of the vehicle in order to detect a valid parking lot. In case a valid parking lot is found, the vehicle stops and performs the required motions for parking backwards autonomously. As the car shall be controlled via TRENTOS, relevant sensor data (e.g. from several proximity sensors) has to be sent from the simulator to the Raspberry Pi 3 B+. The TRENTOS application then calculates the required actuator output (e.g. for the car's steering or braking) and communicates relevant data back to the PC. In addition, the size of the parking lots detected shall be kept in a non-volatile storage. The communication between the Raspberry Pi 3 B+ and the PC is based on Ethernet, utilizing the simulator's API on top. Within the practical project setup, the open source autonomous driving simulator [CARLA](#) will be applied.

The following table provides a detailed overview of the course timeline.

Task	Duration
Practical Project - Autonomous Driving	40h

Train the trainer - Course Description

In the following, descriptions regarding the overall content of the proposed courses will be provided.

Basic Course (80 hours)

This course introduces the fundamental aspects of TRENTOS (<https://www.trentos.de/>), a novel seL4 microkernel based secure embedded operating system developed by HENSOLDT Cyber (<https://hensoldt-cyber.com/>). The course is split in a lecture part, which covers the theoretical background, and a practical part that utilizes different assignments in order to teach participants how to create their very first TRENTOS application, making use of the main TRENTOS building blocks storage, networking and crypto support. For testing the acquired knowledge, two practical projects have to be solved by the participants, which cover the topic device driver implementation/porting utilizing SPI-based network and storage peripherals. Finally, the theoretical knowledge is checked in form of a multiple choice test.

For providing a solid theoretical foundation, the first part of the course covers the basic aspects required for embedded computing in general, providing an introduction to fundamental hardware aspects of the ARM architecture, its respective ISA as well as the final integration of a processor within a system on a chip (SoC). Based on the Broadcom BCM2837, the integration and utilization of a SoC is then exemplarily demonstrated by utilizing the widespread Raspberry Pi 3 B+ single-board computer (SBC). Afterwards, the relevant preparation steps required for further OS utilization are addressed. Relevant aspects include the working with technical reference manuals (TRMs) as well as the utilization of relevant firmware and bootloader components (e.g. U-Boot). Finally, TRENTOS, a secure L4 microkernel based operating system for embedded system development, is introduced. Therefore, the design, architecture and the essential building blocks of the seL4 microkernel are explained. Fundamental aspects like inter-process communication (IPC), the core concept of capabilities, system calls or memory and I/O management are presented in more detail. Based on top of seL4, the CAmkES framework provides respective kernel abstractions that are helpful for further OS and application development, allowing the provision of a suitable userland environment. As a final step, the core concepts and basic software modules of TRENTOS, a secure operating system that utilizes both the seL4 microkernel and the CAmkES framework, are presented. Course participants will understand the inner workings of the OS and get in touch with the main OS aspects storage, networking and crypto support. Additional topics covered will target TRENTOS application and driver development.

Based on several guided practical assignments, course participants will then be taught how to create their first TRENTOS application that can be executed on the Raspberry Pi 3 B+. This will combine a step by step guide (provided and explained by the teacher) and self-contained programming tasks. The assignments will cover the topics

- Basic Case - Part 1: Create a basic TRENTOS application utilizing storage and file system facilities within a virtual execution environment
 - run TRENTOS on QEMU
 - utilize a memory-backed storage backend as a virtual disk
 - provide two partitions
 - understand the TRENTOS storage facilities in combination with the TRENTOS FileSystem API
 - utilize two different file systems (FAT and SPIFFS)
- Basic Case - Part 2: Run TRENTOS on the Raspberry Pi 3 B+
 - understand the TRENTOS build system
 - prepare the SD card resources (e.g. firmware blobs)
 - prepare the wiring/cabling of the SBC
 - receive terminal output from the Raspberry Pi 3 B+ on the PC for debugging purposes
- Assignment 1: Replace the virtual storage backend by a physical storage backend
 - connect a SPI-based NOR flash peripheral device to the Raspberry Pi 3 B+
 - understand the usage of a timer resource
 - understand the concept of simply exchanging underlying storage backends by relying on the abstraction provided by the TRENTOS Storage API
 - reuse the existing TRENTOS SPI NOR flash driver
- Assignment 2: Extend the application via network support utilizing the SBC's internal network interface
 - understand the basic concept of a NIC driver in TRENTOS
 - understand the concept of the TRENTOS network interface
 - understand the utilization of a network stack on top of a NIC driver (both in server and client mode)
 - understand basic network communication on top of the network stack
- Assignment 3: Extend the network part of the application via crypto support
 - understand the basic concept of the TRENTOS Crypto API
 - utilize the TRENTOS TLS API to secure the existing network connection

After having learnt the TRENTOS application layer, the course participants will get in touch with the device driver level. Within two small projects, the participants will write a device driver for two SPI-based peripherals that shall replace the existing network and storage backends used within the aforementioned assignments. Support shall be provided for

- a SPI-based Ethernet peripheral
- a SPI-based FRAM peripheral

The participants will thereby mainly focus on driver porting, which means that an existing open source driver shall be adapted to the respective TRENTOS interfaces (network and storage) and is finally integrated and tested on TRENTOS. The participants shall reuse the underlying GPIO and SPI basic drivers from the TRENTOS SPI NOR flash component and therefore focus on providing the actual peripheral logic.

At the end of the course, the knowledge learnt is checked in form of a multiple choice test.

Course Timeline

The following table provides a detailed overview of the course timeline.

Task	Duration
Lecture	10h
SDK overview	2h
How to write a first TRENTOS application	8h

Assignment 1 (Storage)	4h
Assignment 2 (Network)	12h
Assignment 3 (Crypto)	8h
Practical Project	30h
Multiple Choice Test	2h
Final Integration & Wrap-Up	6h

Extended Course (80 hours)

The extended course builds upon the theoretical knowledge and practical experience acquired within the basic course. Therefore, TRENTOS will be applied within two real world scenarios - industrial automation and autonomous driving. The test environment is provided with the help of two open source simulators, which provide the required virtual sensors and actuators that have to be read and controlled by a respective TRENTOS application.

Practical Project 1 - Industrial Automation

A TRENTOS instance on the Raspberry Pi 3 B+ shall allow for an autonomous sorting setup. It therefore has to control a robotic arm as well as several conveyor belts provided by a robotic simulator that is running on a Linux PC. The robotic arm receives objects of two different colors via a feeding conveyor belt, reads the color of the objects with the help of a virtual color sensor and finally sorts the objects by color via selection between two continuing conveyor belts. As the arm shall be controlled via TRENTOS, relevant sensor data (e.g. from the color sensor) has to be sent from the simulator to the Raspberry Pi 3 B+. The TRENTOS application then calculates the required actuator output (e.g. for the robotic arm's servo motors) and communicates relevant data back to the PC. In addition, the amount of objects sorted by color shall be kept in a non-volatile storage. The communication between the Raspberry Pi 3 B+ and the PC is based on Ethernet, utilizing the simulator's API on top. Within the practical project setup, the open source robotic simulator [CoppeliaSim](#) will be applied.

Practical Project 2 - Autonomous Driving

A TRENTOS instance on the Raspberry Pi 3 B+ shall allow for an autonomous parking setup. It therefore has to control a vehicle in an autonomous driving simulator that runs on a Linux PC. The car drives on a road passing vehicles of different size and distance to each other and measures the distance between itself and the vehicles parking on the road in order to detect free spaces. If a free space is detected, its size has to be measured and compared to the size of the vehicle in order to detect a valid parking lot. In case a valid parking lot is found, the vehicle stops and performs the required motions for parking backwards autonomously. As the car shall be controlled via TRENTOS, relevant sensor data (e.g. from several proximity sensors) has to be sent from the simulator to the Raspberry Pi 3 B+. The TRENTOS application then calculates the required actuator output (e.g. for the car's steering or braking) and communicates relevant data back to the PC. In addition, the size of the parking lots detected shall be kept in a non-volatile storage. The communication between the Raspberry Pi 3 B+ and the PC is based on Ethernet, utilizing the simulator's API on top. Within the practical project setup, the open source autonomous driving simulator [CARLA](#) will be applied.

Course Timeline

The following table provides a detailed overview of the course timeline.

Task	Duration
Practical Project 1 - Industrial Automation	40h
Practical Project 2 - Autonomous Driving	40h

Tasks

Ticket Type	Ticket Name	Ticket Description	seL4 Foundation Dependencies	Current State	Expected time	
Story	Align scope of the trainings	Align the scope of the planned trainings for - seL4 Foundation - Train the trainer with regard to the topics - seL4 - CAMkes - TRENTOS				
Epic	Prepare the basic course for "train the trainer"					

Story	Revise TUM lecture material to be reused for the basic course (train the trainer)	The existing TUM lecture material can be reused as a baseline for the lecture units to be provided for train the trainer. Nevertheless, the material has to be adapted accordingly, as all TUM-specific notes have to be removed.	SEOS-2175 - Getting issue details... STATUS		8h	
Story	Create SDK overview	Create an SDK overview that provides more detailed information on - the RPi background (especially the RPi 3 B+) - the individual SDK components The SDK overview basically transfers the theoretical SDK explanations from the lecture into a step by step setup guide that will be done live and together with the participants during the course.			24h	
SEOS-2707 - Getting issue details... STATUS	Revise the Basic Case	Adapt the basic case to TRENTOS v1.2			4h	
SEOS-2708 - Getting issue details... STATUS	Revise Assignment 1 (Storage)	Adapt assignment 1 to TRENTOS v1.2			4h	
SEOS-2709 - Getting issue details... STATUS	Revise Assignment 2 (Networking)	Adapt assignment 2 to TRENTOS v1.2			8h	
SEOS-2710 - Getting issue details... STATUS	Create Assignment 3 (Crypto)	Create an additional assignment which covers the TRENTOS Crypto API. Reuse the TRENTOS TLS API, which uses the Crypto API underneath, to secure the network communication created in assignment 2.			40h	
Story	Prepare the SPI-based Ethernet adapter project	Provide a basic guide for the course participants to let them do the actual porting. Prepare and offer a sample solution for the task.			24h	
Story	Prepare the SPI-based FRAM adapter project	Provide a basic guide for the course participants to let them do the actual porting. Prepare and offer a sample solution for the task.			24h	
Story	Prepare the multiple choice questions for the final test	Prepare about 100 questions, from which a part of them will be selected for the test.			40h	
Epic	Prepare the extended course for "train the trainer"					
Story	Prepare a robotic simulator use case setup	- Provide a basic guide for the course participants to let them do the actual implementation - Provide some kind of skeleton the participants can build on (e.g. the simulator environment), so they don't have to deal with aspects that are not required - Prepare and offer a sample solution for the task			60h	
Story	Prepare an autonomous driving simulator use case setup	- Provide a basic guide for the course participants to let them do the actual implementation - Provide some kind of skeleton the participants can build on (e.g. the simulator environment), so they don't have to deal with aspects that are not required - Prepare and offer a sample solution for the task			60h	

Story	Prepare a hardware crypto extension use case setup	Idea: integration of a dedicated, simple crypto coprocessor (e.g. https://learn.sparkfun.com/tutorials/cryptographic-co-processor-atecc508a-qwiic-hookup-guide/all) • Provide a basic guide for the course participants to let them do the actual implementation • Provide some kind of skeleton the participants can build on, so they don't have to deal with aspects that are not required • Prepare and offer a sample solution for the task			120h	
-------	--	--	--	--	------	--